



If an $n \times n$ distance matrix D is non-additive, then we will instead look for a weighted tree T whose distances between leaves approximate the entries in D . To this end, we would like for T to minimize the **sum of squared errors** $Discrepancy(T, D)$, which is given by the formula

$$Discrepancy(T, D) = \sum_{1 \leq i < j \leq n} (d_{i,j}(T) - D_{i,j})^2.$$

Least Squares Distance-Based Phylogeny Problem: *Given a distance matrix, find the tree that minimizes the sum of squared errors.*

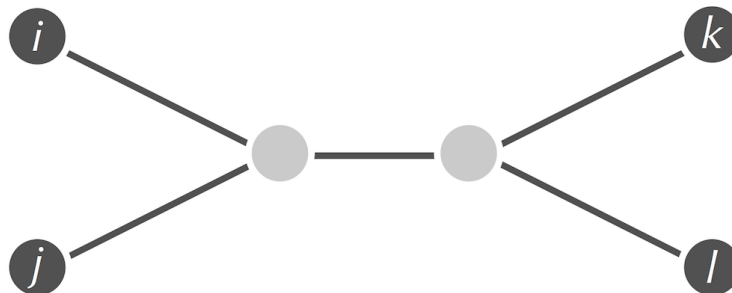
Input: An $n \times n$ distance matrix D .

Output: A weighted tree T with n leaves minimizing $Discrepancy(T, D)$ over all weighted trees with n leaves.





Exercise Break: Let T be the tree shown below.



Given the non-additive 4×4 distance matrix D below, find the lengths of edges in this tree that minimize $\text{Discrepancy}(T, D)$.

	i	j	k	l
i	0	3	4	3
j	3	0	4	5
k	4	4	0	2
l	3	5	2	0



It turns out that for a specific tree T , it is easy to find edge weights in T minimizing $Discrepancy(T, D)$. Yet our ability to minimize the sum of squared errors for a specific tree does not imply that we can efficiently solve the Least Squares Distance-Based Phylogeny Problem, since the number of different trees grows very quickly as the number of leaves in the tree increases. In fact, the Least Squares Distance-Based Phylogeny winds up being *NP*-Complete, and so we must abandon the hope of designing a fast algorithm to find a tree that best fits a non-additive matrix. In the next two sections, we will explore heuristics for constructing trees from non-additive matrices that solve this problem approximately.





Biologists often assume that every internal node in an evolutionary tree corresponds to a species that underwent a **speciation event**, splitting one ancestral species into two descendants. Note that every internal node in the tree in the figure below (corresponding to a speciation event) has degree 3. We therefore define an **unrooted binary tree** as a tree where every node has degree equal to either 1 or 3.

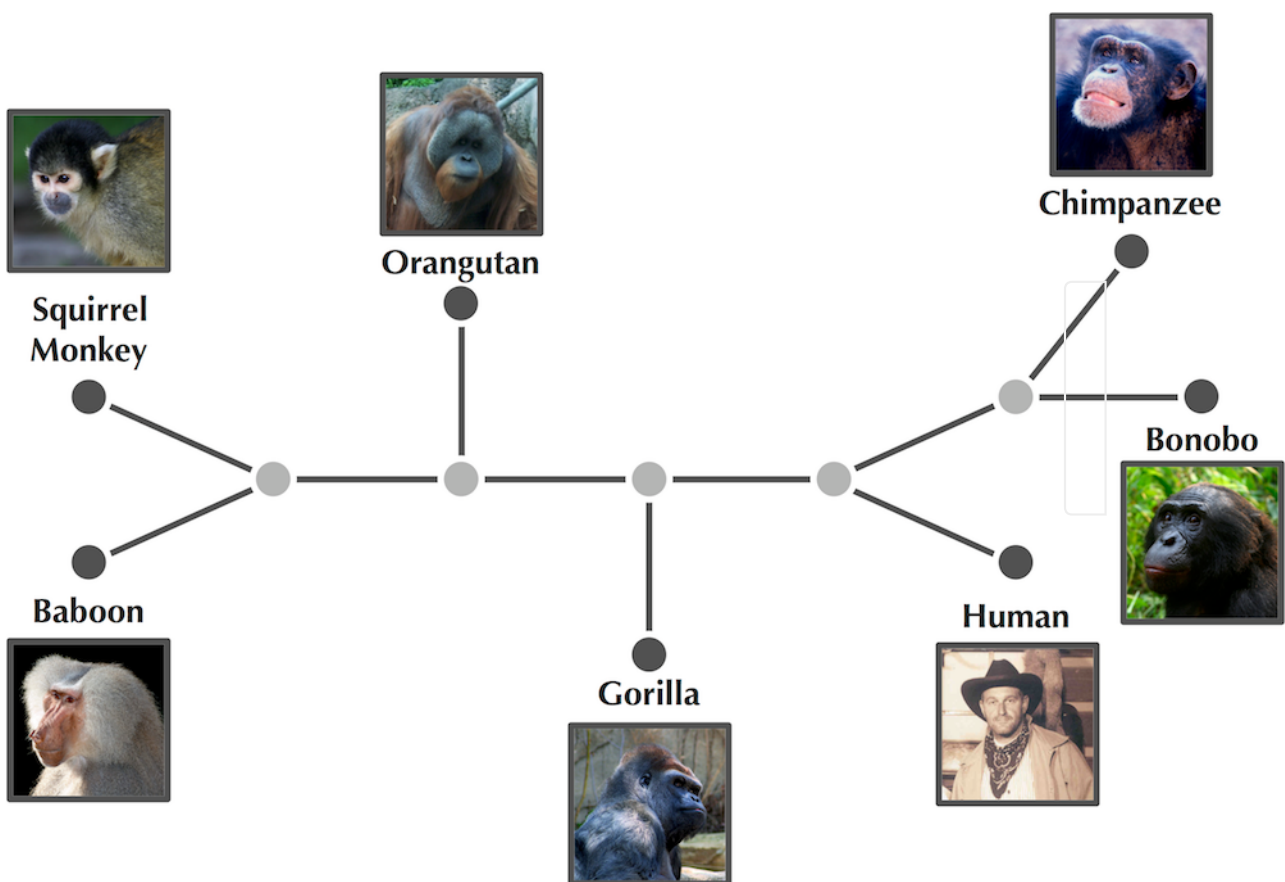


Figure: An unrooted binary tree modeling the primate phylogeny.

Exercise Break: Prove that every unrooted binary tree with n leaves has $n - 2$ internal nodes (and thus $(2n - 3)$ edges).



A **rooted binary tree** is an unrooted binary tree that has a root (of degree 2) placed on one of its edges; in other words, we replace an edge (v, w) with a root and draw edges connecting the root to each of v and w .

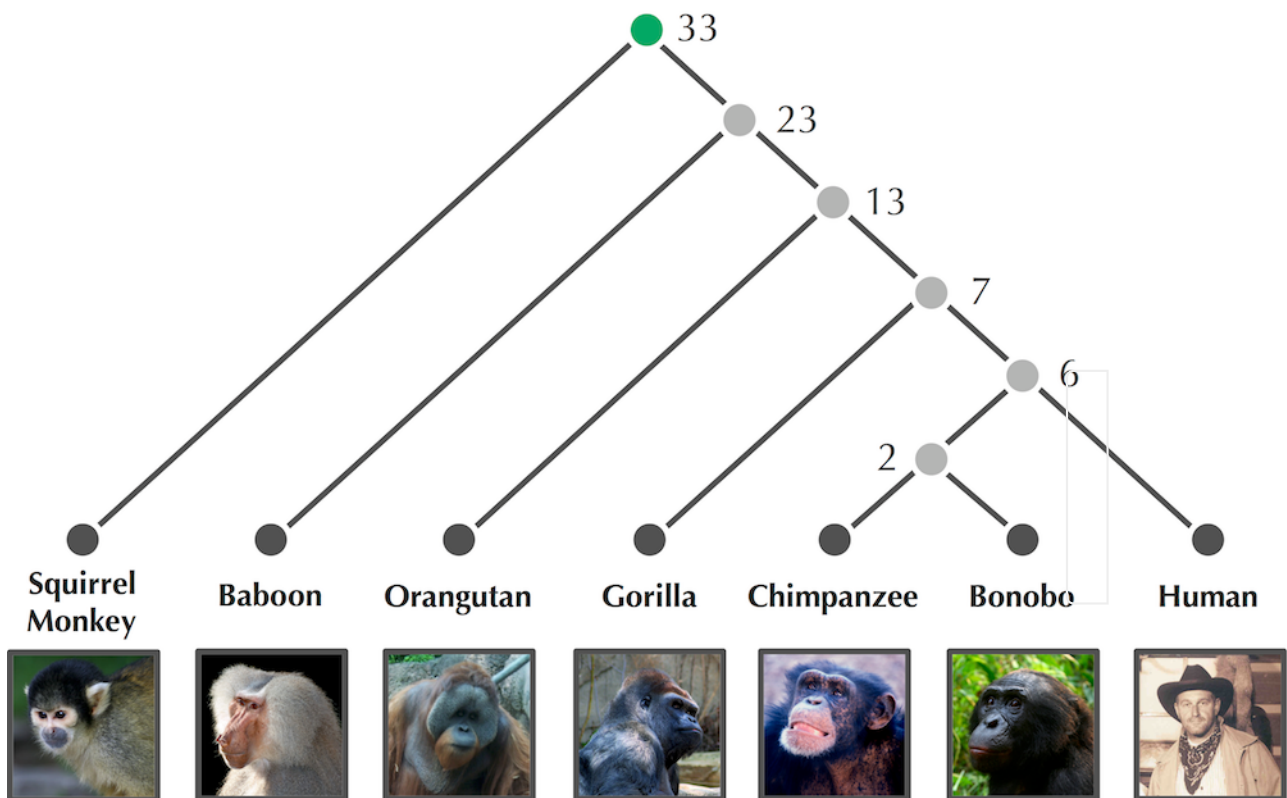


Figure: Placing a root on the squirrel monkey's limb results in a rooted binary tree. The number at each node corresponds to the estimate of the number of million years ago that the divergence at that node occurred. Image courtesies: Julie Langford (squirrel monkey), André Karwath (old world monkey), Frans de Waal (chimp), Kabir Bakie (gorilla).



If we had a **molecular clock** measuring evolutionary time, then we could assign an **age** to every node v in a rooted binary tree (denoted $Age(v)$), where all of the leaves of the tree have age 0 because they correspond to present-day species. We could then define the weight of an edge (v, w) in the tree as the difference $Age(v) - Age(w)$. Consequently, the length of a path between the root and any node would be equal to the difference between their ages. Such a tree, in which the distance from the root to any leaf is the same, is called **ultrametric** (see figure below).

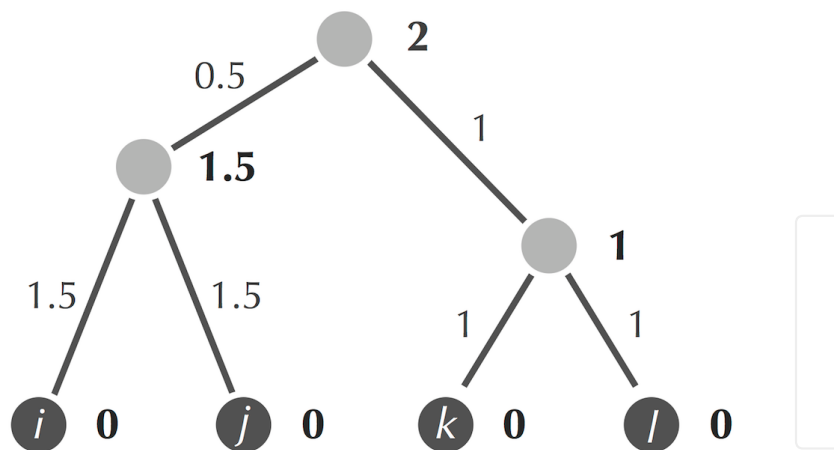


Figure: An ultrametric tree. Each node has been assigned an age (in boldface), where each leaf has age equal to zero. The length of each edge is equal to the difference between the ages of the nodes that the edge connects, and the distance from the root to any leaf is equal to 2, the age of the root.



Our aim is to derive an ultrametric tree that explains a given distance matrix. **UPGMA** (which stands for **U**nweighted **P**air **G**roup **M**ethod with **A**rithmetic Mean) is a simple clustering heuristic that introduces a hypothetical molecular clock for constructing an ultrametric evolutionary tree. You can learn more about clustering in a later chapter.

Given an $n \times n$ matrix D , **UPGMA** (which is illustrated in the figure on the next step) first forms n trivial clusters, each containing a single leaf. The algorithm then looks for a pair of “closest” clusters. To clarify the notion of closest clusters, **UPGMA** defines the distance between clusters C_1 and C_2 as the average pairwise distance between elements of C_1 and C_2 :

$$D_{C_1, C_2} = \frac{\sum_{i \in C_1} \sum_{j \in C_2} D_{i,j}}{(|C_1| \cdot |C_2|)}.$$

In this equation, the notation $|C|$ denotes the number of leaves in cluster C .



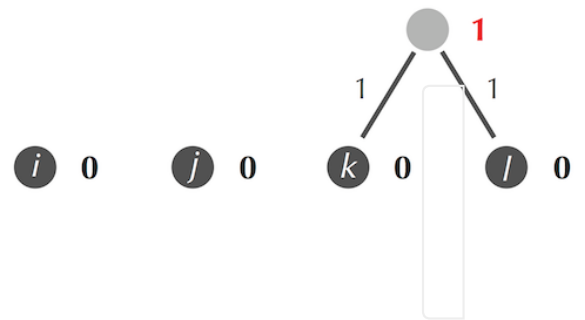
Ultrametric Evolutionary Trees | Step 5

10 / 10

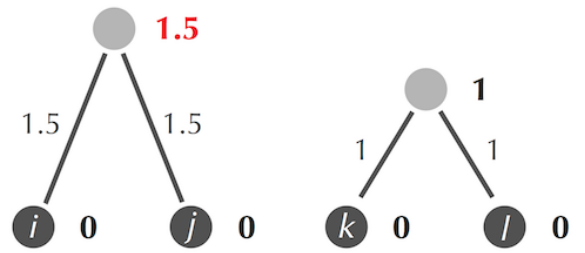
	<i>i</i>	<i>j</i>	<i>k</i>	<i>l</i>
<i>i</i>	0	3	4	3
<i>j</i>	3	0	4	5
<i>k</i>	4	4	0	2
<i>l</i>	3	5	2	0



	<i>i</i>	<i>j</i>	<i>k</i>	<i>l</i>
<i>i</i>	0	3	4	3
<i>j</i>	3	0	4	5
<i>k</i>	4	4	0	2
<i>l</i>	3	5	2	0



	<i>i</i>	<i>j</i>	<i>{k, l}</i>
<i>i</i>	0	3	3.5
<i>j</i>	3	0	4.5
<i>{k, l}</i>	3.5	4.5	0



	<i>{i, j}</i>	<i>{k, l}</i>
<i>{i, j}</i>	0	4
<i>{k, l}</i>	4	0

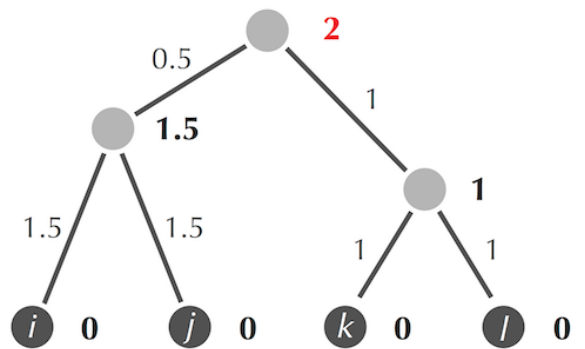


Figure: Tree reconstruction with **UPGMA** for the non-additive distance matrix we have encountered previously. **UPGMA** begins with forming one cluster for each leaf. In each step, it identifies the two closest clusters C_1 and C_2 , merge them into a new node C , and connect C to C_1 and C_2 by directed edges. The age of C is set equal to $D_{C_1, C_2} / 2$. We then iterate this process until only a single cluster remains, which must be the root.



Once **UPGMA** has identified a pair of closest clusters C_1 and C_2 , it merges them into a cluster C with $|C_1| + |C_2|$ elements and then creates a node for C , which it connects to each of C_1 and C_2 by a directed edge. The age of C is set to be $D_{C_1, C_2} / 2$. **UPGMA** then iterates this process of merging the two closest clusters until only a single cluster remains, which corresponds to the root.

Exercise Break: Prove that after merging clusters C_1 and C_2 into a cluster C , the distance between C and another cluster C_m is equal to $(D_{C_1, C_m} \cdot |C_1| + D_{C_2, C_m} \cdot |C_2|) / (|C_1| + |C_2|)$.





The pseudocode for **UPGMA** is shown below.

UPGMA(D, n)

form n clusters, each containing a single element i (for i from 1 to n)

construct a graph T by assigning a node to each cluster (without adding any edges)

for every node v in T

$Age(v) \leftarrow 0$

while there is more than one cluster

 find two closest clusters C_1 and C_2 (break ties arbitrarily)

 merge C_1 and C_2 into a new cluster C

 add a new node C to T and connect it to nodes C_1 and C_2 by directed edges

$Age(C) \leftarrow D_{C_1, C_2} / 2$

 remove rows and columns of D corresponding to C_1 and C_2

 add a row and column to D for C by recomputing D_{C, C^*} for each $C^* \neq C$

$root \leftarrow$ the node in T corresponding to the cluster C

for each edge (v, w) in T

$Length(v, w) \leftarrow Age(v) - Age(w)$

return T

**CODE CHALLENGE:** Implement **UPGMA**.

Input: An integer n followed by a space separated $n \times n$ distance matrix.

Output: An adjacency list for the ultrametric tree returned by **UPGMA**. Edge weights

should be accurate to two decimal places (answers in the sample dataset below are provided to three decimal places).

Note on formatting: The adjacency list must have consecutive integer node labels starting from 0. The n leaves must be labeled 0, 1, ..., $n - 1$ in order of their appearance in the distance matrix. Labels for internal nodes may be labeled in any order but must start from n and increase consecutively.

Extra Dataset <http://bioinformaticsalgorithms.com/data/extradatasets/evolution/UPGMA.txt>

Sample Input:

```
4
0      20      17      11
20      0      20      13
17      20      0      10
11      13      10      0
```

Sample Output:

```
0->5:7.000
1->6:8.833
2->4:5.000
3->4:5.000
4->2:5.000
4->3:5.000
4->5:2.000
5->0:7.000
5->4:2.000
5->6:1.833
6->5:1.833
6->1:8.833
```

Time Limit: 5 mins

To solve this problem please visit stepic.org/lesson/-10332/step/8



Exercise Break: Apply **UPGMA** to the (non-additive) coronavirus distance matrix reproduced below.

	Cow	Pig	Horse	Mouse	Dog	Cat	Turkey	Civet	Human
Cow	0	295	300	524	1077	1080	978	941	940
Pig	295	0	314	487	1071	1088	1010	963	966
Horse	300	314	0	472	1085	1088	1025	965	956
Mouse	524	487	472	0	1101	1099	1021	962	965
Dog	1076	1070	1085	1101	0	818	1053	1057	1054
Cat	1082	1088	1088	1098	818	0	1070	1085	1080
Turkey	976	1011	1025	1021	1053	1070	0	963	961
Civet	941	963	965	962	1057	1085	963	0	16
Human	940	966	956	965	1054	1080	961	16	0

Download Matrix

(http://bioinformaticsalgorithms.com/data/realdatasets/Evolution/coronavirus_distance_matrix_nonadditive.txt)



UPGMA offers a step forward from **AdditivePhylogeny**, since it can analyze non-additive distance matrices. The figure below shows the result of applying **UPGMA** to the coronavirus distance matrix. However, the first step that **UPGMA** takes is to merge the two leaves i and j with minimum distance $D_{i,j}$ into a single cluster. We have already seen that the smallest element in the distance matrix does not necessarily correspond to a pair of neighboring leaves! This is a concern, since if **UPGMA** generates incorrect trees from additive matrices, then it is not an ideal heuristic for evolutionary tree construction from non-additive matrices. Can we find an algorithm that always identifies neighboring leaves in an additive distance matrix but also performs well on a non-additive distance matrix?

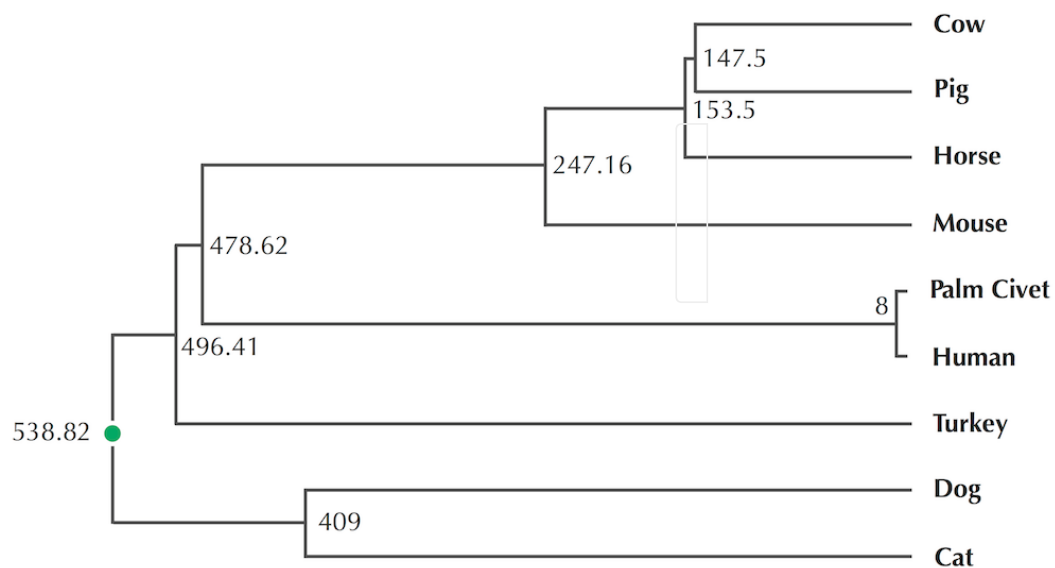


Figure: The ultrametric tree of coronaviruses created by **UPGMA** using the coronavirus distance matrix. The root is shown in green.



Transforming a distance matrix into its neighbor-joining matrix

In 1987, Naruya Saitou and Masatoshi Nei developed the **neighbor-joining algorithm** for evolutionary tree reconstruction. Given an additive distance matrix, this algorithm, which we call **NeighborJoining**, finds a pair of neighboring leaves and substitutes them by a single leaf, thus reducing the size of the tree. **NeighborJoining** can thus recursively construct a tree fitting the additive matrix. This algorithm also provides a heuristic for non-additive distance matrices that performs well in practice.

The central idea of **NeighborJoining** is that although finding a minimum element in a distance matrix D is not guaranteed to yield a pair of neighbors in $Tree(D)$, we can convert D into a different matrix whose minimum element does yield a pair of neighbors. First, given an $n \times n$ distance matrix D , we define $TotalDistance_D(i)$ as the sum $\sum_{1 \leq k \leq n} D_{i,k}$ of distances from leaf i to all other leaves. The **neighbor-joining matrix** D^* (see below) is defined such that for any i and j , $D^*_{ii} = 0$ and

$$D^*_{ij} = (n - 2) \cdot D_{ij} - TotalDistance_D(i) - TotalDistance_D(j).$$

<i>D</i>		<i>i</i>	<i>j</i>	<i>k</i>	<i>l</i>	<i>TotalDistance_D</i>	<i>D*</i>		<i>i</i>	<i>j</i>	<i>k</i>	<i>l</i>
	<i>i</i>	0	13	21	22	56		<i>i</i>	0	-68	-60	-60
	<i>j</i>	13	0	12	13	38		<i>j</i>	-68	0	-60	-60
	<i>k</i>	21	12	0	13	46		<i>k</i>	-60	-60	0	-68
	<i>l</i>	22	13	13	0	48		<i>l</i>	-60	-60	-68	0



NeighborJoining (illustrated on the next step) is the most widely used method for evolutionary tree reconstruction; the paper that introduced it is one of the most cited in all of science, with over 30,000 citations. Yet this algorithm is non-intuitive: the above formula for computing the matrix D^* probably looks like witchcraft to you. In fact, despite having flawless intuition, Saitou and Nei *never proved* that their algorithm correctly solves the Distance-Based Phylogeny Problem for additive matrices! It took researchers another decade to prove the following theorem, whose proof we have passed to [DETOUR: Why does the neighbor-joining algorithm find neighboring leaves?](http://stepic.org/lesson/Detour-Why-does-the-neighbor-joining-algorithm-find-neighboring-leaves/) ([http://stepic.org/lesson/Detour-Why-does-the-neighbor-joining-algorithm-find-neighboring-leaves?](http://stepic.org/lesson/Detour-Why-does-the-neighbor-joining-algorithm-find-neighboring-leaves/))

Neighbor-Joining Theorem: Given an additive matrix D , the smallest element D_{ij}^* of its neighbor-joining matrix D^* reveals a pair of neighboring leaves i and j in $Tree(D)$.

If $n = 2$, then $NeighborJoining(D, n)$ returns the tree consisting of a single edge of length $D_{1,2}$. If $n > 2$, then it selects the minimum element in the neighbor-joining matrix, replaces the neighboring leaves i and j with a new leaf m , and then computes the distance from m to any other leaf k according to the equation

$$D_{k,m} = (1/2)(D_{k,i} + D_{k,j} - D_{i,j}).$$

which is motivated by **AdditivePhylogeny**. This equation allows us to replace an $n \times n$ matrix D with an $(n - 1) \times (n - 1)$ matrix D' in which i and j have been replaced by m . By recursively applying $NeighborJoining$ to D' , we obtain an evolutionary tree with $n - 1$ leaves. We then add two limbs starting at node m , one ending in leaf i and the other ending in leaf j . We set

$$\Delta_{ij} = (TotalDistance_D(i) - TotalDistance_D(j)) / (n - 2)$$

and assign

$$LimbLength(i) = (1/2) (D_{i,j} + \Delta_{ij})$$

$$LimbLength(j) = (1/2) (D_{i,j} - \Delta_{ij})$$

To see where these formulas come from, see [DETOUR: Computing Limb Lengths in the Neighbor-Joining Algorithm](http://stepic.org/lesson/Detour-Computing-Limb-Lengths-in-the-Neighbor-Joining-Algorithm-10343/) (<http://stepic.org/lesson/Detour-Computing-Limb-Lengths-in-the-Neighbor-Joining-Algorithm-10343/>).

Exercise Break: Prove that for any i and j between 1 and n , both $(1/2) (D_{i,j} + \Delta_{ij})$ and $(1/2) (D_{i,j} - \Delta_{ij})$ are non-negative.



	i	j	k	l	$TotalDistance_D$
i	0	13	21	22	56
j	13	0	12	13	38
k	21	12	0	13	46
l	22	13	13	0	48



	i	j	k	l
i	0	-68	-60	-60
j	-68	0	-60	-60
k	-60	-60	0	-68
l	-60	-60	-68	0



	m	k	l	$TotalDistance_D$
m	0	10	11	21
k	10	0	13	23
l	11	13	0	24



	m	k	l
m	0	-34	-34
k	-34	0	-34
l	-34	-34	0



	m	a
m	0	4
a	4	0

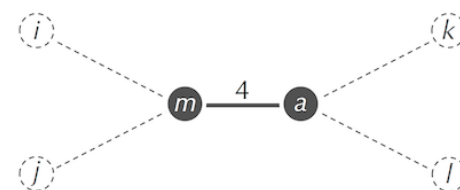
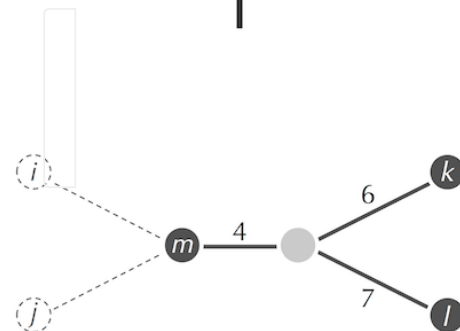
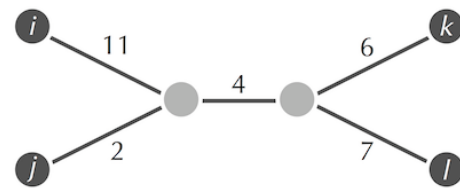


Figure: (Top left) A 4×4 additive distance matrix D we have considered before along with the array $TotalDistance_D$. $D_{j,k}$ (shown in blue) is a minimum element of D , but as it turns out, leaves j and k are not neighbors in $Tree(D)$. Moving down the left side, we construct the neighbor-joining matrix D^* and find that D_{ij}^* (red) is a minimum element of D . We then transform the initial 4×4 distance matrix into a 3×3 distance matrix by replacing i and j with a single leaf m and updating distances from m to other leaves according to the equation $D_{k,m} = (1/2)(D_{k,i} + D_{k,j} - D_{i,j}) = (1/2)(21+12 - 13) = 10$. All elements are minimum elements in the new matrix D , so we arbitrarily select k and l as neighbors to be replaced with a single leaf a . The resulting 2×2 matrix corresponds to a tree with a single edge connecting m and a . We then work our way up the right side, adding pairs of neighbors back into the tree at each step using previously computed formulas for limb lengths. (Top right) The tree $Tree(D)$ fitting the original matrix D .



The pseudocode below summarizes the neighbor-joining algorithm.

NeighborJoining(D, n)

if $n = 2$

$T \leftarrow$ tree consisting of a single edge of length $D_{1,2}$

return T

$D^* \leftarrow$ neighbor-joining matrix constructed from D

find elements i and j such that D^*_{ij} is a minimum non-diagonal element of D^*

$\Delta \leftarrow (TotalDistance_D(i) - TotalDistance_D(j)) / (n - 2)$

$limbLength_i \leftarrow (1/2)(D_{ij} + \Delta)$

$limbLength_j \leftarrow (1/2)(D_{ij} - \Delta)$

add a new row/column m to D so that $D_{k,m} = D_{m,k} = (1/2)(D_{k,i} + D_{k,j} - D_{i,j})$ for any k

remove i -th row and column as well as j -th row and column from D

$T \leftarrow NeighborJoining(D, n - 1)$

add two new limbs (connecting node m with leaves i and j) to the tree T

assign length $limbLength_i$ to $Limb(i)$

assign length $limbLength_j$ to $Limb(j)$

return T



Exercise Break: Before implementing the neighbor-joining algorithm, try applying it to the additive and non-additive distance matrices shown below.

	<i>i</i>	<i>j</i>	<i>k</i>	<i>l</i>
<i>i</i>	0	13	21	22
<i>j</i>	13	0	12	13
<i>k</i>	21	12	0	13
<i>l</i>	22	13	13	0

	<i>i</i>	<i>j</i>	<i>k</i>	<i>l</i>
<i>i</i>	0	3	4	3
<i>j</i>	3	0	4	5
<i>k</i>	4	4	0	2
<i>l</i>	3	5	2	0

**CODE CHALLENGE:** Implement **NeighborJoining**.

Input: An integer n , followed by an $n \times n$ distance matrix.

Output: An adjacency list for the tree resulting from applying the neighbor-joining algorithm. Edge-weights should be accurate to two decimal places (they are provided to three decimal places in the sample output below).

Note on formatting: The adjacency list must have consecutive integer node labels starting from 0. The n leaves must be labeled 0, 1, ..., $n - 1$ in order of their appearance in the distance matrix. Labels for internal nodes may be labeled in any order but must start from n and increase consecutively.

Extra Dataset (http://bioinformaticsalgorithms.com/data/extradata/evolution/Neighbor_Joining.txt)

Sample Input:

4			
0	23	27	20
23	0	30	28
27	30	0	30
20	28	30	0

Sample Output:

```
0->4:8.000
1->5:13.500
2->5:16.500
3->4:12.000
4->5:2.000
4->0:8.000
4->3:12.000
5->1:13.500
5->2:16.500
5->4:2.000
```

Time Limit: 5 mins

To solve this problem please visit stepic.org/lesson/-10333/step/6



Exercise Break: Apply **NeighborJoining** to the nonadditive coronavirus distance matrix, reproduced below.

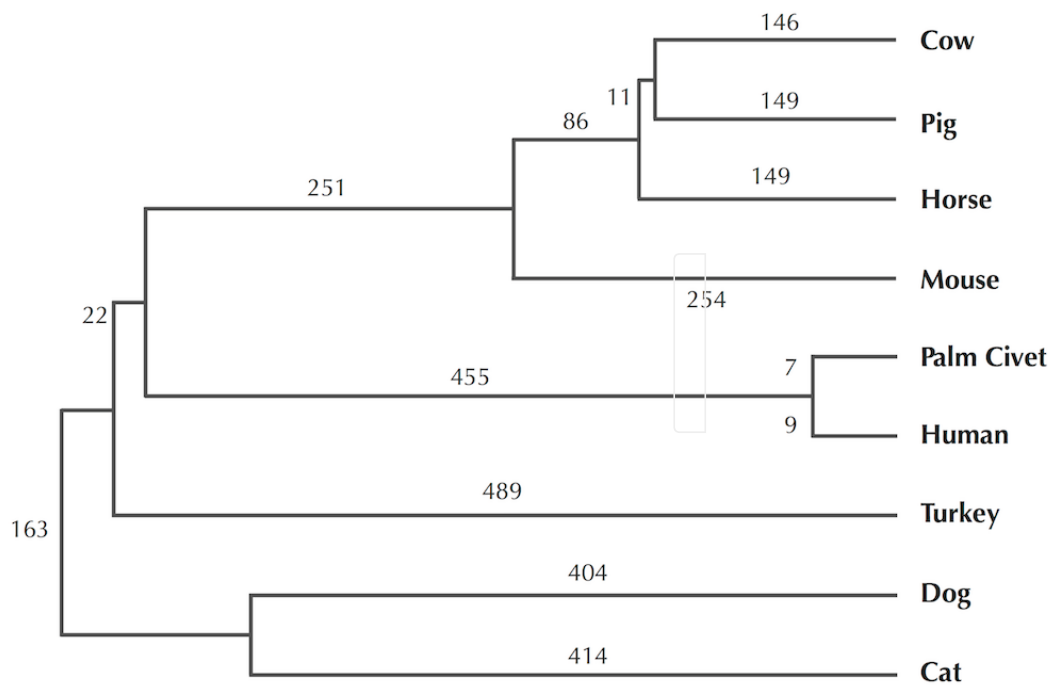
	Cow	Pig	Horse	Mouse	Dog	Cat	Turkey	Civet	Human
Cow	0	295	300	524	1077	1080	978	941	940
Pig	295	0	314	487	1071	1088	1010	963	966
Horse	300	314	0	472	1085	1088	1025	965	956
Mouse	524	487	472	0	1101	1099	1021	962	965
Dog	1076	1070	1085	1101	0	818	1053	1057	1054
Cat	1082	1088	1088	1098	818	0	1070	1085	1080
Turkey	976	1011	1025	1021	1053	1070	0	963	961
Civet	941	963	965	962	1057	1085	963	0	16
Human	940	966	956	965	1054	1080	961	16	0

Download Matrix (http://bioinformaticsalgorithms.com/data/realdatasets/Evolution/coronavirus_distance_matrix_nonadditive.txt)



Analyzing coronaviruses with the neighbor-joining algorithm

The figure below shows the neighbor-joining tree of coronaviruses isolated from different animals based on the nonadditive distance matrix from the previous step. (Edge-weights are rounded to the nearest integer.)





We can also apply **NeighborJoining** to the distance matrix of SARS-CoV variants isolated from various human carriers, in addition to a coronavirus from palm civet (see the figure below).

	Guangzhou Dec. 16, 2002	Zhonghan Dec.16, 2002	Guangzhou Jan. 24, 2003	Guangzhou Jan. 31, 2003	Guangzhou Feb. 18, 2003	Hong Kong Feb. 18, 2003	Hanoi Feb. 26, 2003	Toronto Feb. 17, 2003	Hong Kong March 15, 2003	Palm civet
Guangzhou	0	4	12	8	9	9	12	12	11	3
Zhonghan	4	0	10	6	7	7	10	10	9	3
Guangzhou	12	10	0	4	5	3	2	2	1	11
Guangzhou	8	6	4	0	3	1	4	4	3	7
Guangzhou	9	7	5	3	0	2	5	5	4	8
Hong Kong	9	7	3	1	2	0	3	3	2	8
Hanoi	12	10	2	4	5	3	0	2	1	11
Toronto	12	10	2	4	5	3	2	0	1	11
Hong Kong	11	9	1	3	4	2	1	1	0	10
Palm civet	3	3	11	7	8	8	11	11	10	0

The SARS-CoV strain labeled “Hanoi” in this figure was taken from Carlo Urbani, an Italian physician who worked for the World Health Organization. In February 2003, Urbani was called into a Hanoi hospital to examine a patient who had fallen ill with what local doctors believed to be a bad case of influenza; these doctors were afraid that it might be bird flu. In fact, the patient was the American man who had stayed across the hall from Liu Jianlun at the Metropole Hotel just one week earlier. Fortunately, Urbani was quick to realize that the disease was not influenza, and he became the first physician to raise the alarm to public health officials. Yet rather than leaving Hanoi, he demanded that he remain there in order to oversee quarantine procedures. In an argument with his wife, who scolded him for risking his life to treat sick patients. Urbani replied, “What am I here for? Answering e-mails, going to cocktail parties and pushing paper?” Urbani would ultimately lose his life to SARS a month later. But his sacrifice also helped begin the massive worldwide action against this disease that may well have saved millions of lives.

Limitations of distance-based approaches to evolutionary tree reconstruction

Although distance-based tree reconstruction has successfully resolved questions about the origin and spread of SARS, many evolutionary controversies cannot be resolved by using a distance matrix. For example, when we convert each pair of rows of a multiple alignment into a distance value, we lose information contained in the alignment. As a result, distance-based phylogeny does not allow us to reconstruct ancestral sequences of Spike proteins (corresponding to internal nodes of the Spike protein phylogeny), which could lead us to believe that such molecular paleontology is impossible. Therefore, a superior approach to evolutionary tree reconstruction would be to somehow use the alignment directly, without first converting it to a distance matrix.





We stated in the main text that if the distance matrix D is additive, then there exists a unique simple tree $Tree(D)$ fitting this matrix. This is not quite true, since if a simple tree fits D and has an internal edge of weight zero, then we can easily remove this edge by “gluing” together the nodes that it connects (see figure below). Thus, we will make a further assumption that not only is $Tree(D)$ simple, but that it contains no internal edges of length zero.

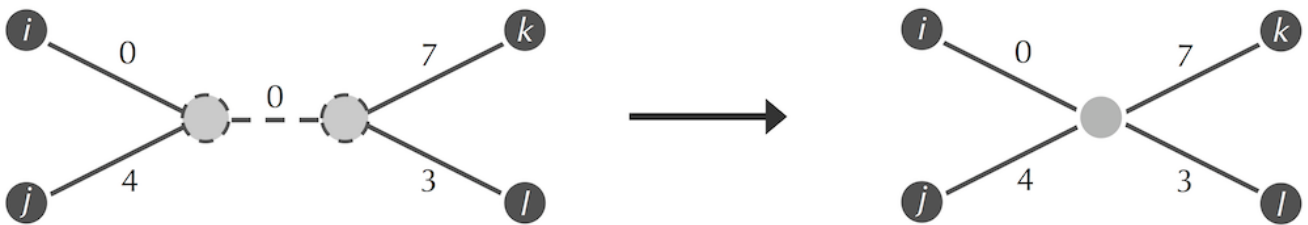


Figure: Gluing the nodes at the endpoints of an internal edge having zero length (shown by the dashed edge).



Now, we can rewrite the neighbor-joining matrix formula as follows:

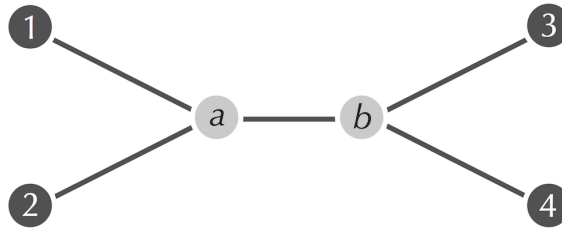
$$D_{ij}^* = (n - 2) \cdot D_{ij} - \text{TotalDistance}_D(i) - \text{TotalDistance}_D(j) = (n - 2) \cdot D_{ij} - \sum_{1 \leq k \leq n} D_{i,k} - \sum_{1 \leq k \leq n} D_{j,k}$$

Each element in this formula further breaks down as a sum of edge weights in $\text{Tree}(D)$. For example, for the first tree in the figure below,

$$\begin{aligned} D_{1,2}^* &= 2 \cdot d_{1,2} - (d_{1,3} + d_{1,4} + d_{1,2}) - (d_{2,3} + d_{2,4} + d_{1,2}) \\ &= 2 \cdot (d_{1,a} + d_{a,2}) - ((d_{1,a} + d_{a,b} + d_{b,3}) + (d_{1,a} + d_{a,b} + d_{b,4}) + (d_{1,a} + d_{a,2})) - ((d_{2,a} + d_{a,b} + d_{b,3}) + (d_{2,a} + d_{a,b} + d_{b,4}) + (d_{1,a} + d_{a,2})) \\ &= -2 \cdot d_{1,a} - 2 \cdot d_{2,a} - 2 \cdot d_{b,3} - 2 \cdot d_{b,4} - 4 \cdot d_{a,b} \end{aligned}$$

and

$$\begin{aligned} D_{1,3}^* &= 2 \cdot d_{1,3} - (d_{1,2} + d_{1,4} + d_{1,3}) - (d_{3,4} + d_{3,2} + d_{3,1}) \\ &= 2 \cdot (d_{1,a} + d_{a,b} + d_{b,3}) - ((d_{1,a} + d_{a,2}) + (d_{1,a} + d_{a,b} + d_{b,4}) + (d_{1,a} + d_{a,b} + d_{b,3})) - ((d_{3,b} + d_{b,4}) + (d_{3,b} + d_{b,a} + d_{b,2}) + (d_{3,b} + d_{b,a} + d_{a,1})) \\ &= -2 \cdot d_{1,a} - 2 \cdot d_{2,a} - 2 \cdot d_{b,3} - 2 \cdot d_{b,4} - 2 \cdot d_{a,b} \end{aligned}$$



Note that the expressions for $D_{1,2}^*$ and $D_{1,3}^*$ are nearly identical, with only the coefficients of $d_{a,b}$ differing (highlighted in blue above). Since $D_{1,2}^* - D_{1,3}^* = -2 \cdot d_{a,b} < 0$, the neighbor-joining algorithm will prefer the smaller $D_{1,2}^*$ over $D_{1,3}^*$.



Given an edge e in $Tree(D)$, the **multiplicity** of this edge in D^*_{ij} , denoted $Multiplicity_{ij}(e)$, is the coefficient of d_e in the expression for D^*_{ij} . For example, for the edge $e = (a, b)$ in the first tree in Figure 1.35, $Multiplicity_{1,2}(e) = -4$, and $Multiplicity_{1,3}(e) = -2$. The following result shows that all limbs in $Tree(D)$ have the same multiplicity.

Lemma: For an additive distance matrix D and any pair of leaves i and j in $Tree(D)$, $Multiplicity_{ij}(e)$ is equal to -2 for any limb e in $Tree(D)$.

Proof: If a limb e is not the limb of leaf i or leaf j , then it is counted zero times in $(n - 2) \cdot D_{ij}$, once in $TotalDistance_D(i)$ and once in $TotalDistance_D(j)$, making its multiplicity -2 . On the other hand, if e is the limb of i or j (let's say i), then it is counted $n - 2$ times in $(n - 2) \cdot D_{ij}$, $n - 1$ times in $TotalDistance_D(i)$, and once time in $TotalDistance_D(j)$. Therefore, its multiplicity is $n - 2 - (n - 1) - 1 = -2$. $\square$

This lemma implies that regardless of which pair of leaves i and j that we choose, the limbs of $Tree(D)$ will all have the same contribution to the computation of D^*_{ij} . As a result, only the multiplicities of internal edges of $Tree(D)$ differentiate values of the matrix D^* . Can we determine these multiplicities?

Exercise Break: Prove that for any i and j , and for any internal edge e in $Tree(D)$, $Multiplicity_{ij}(e) \leq -2$.



Not only does the multiplicity of each internal edge not exceed -2, but we also have a condition determining when an internal edge will have multiplicity equal to -2. To derive this condition, first note that the removal of an internal edge e disconnects any tree into two subtrees. If e lies on the (unique) path connecting leaves i and j in $Tree(D)$, denoted $Path(i, j)$, then i and j belong to different subtrees, denoted T_i and T_j , respectively; otherwise, i and j belong to the same subtree. In the latter case, we denote the number of leaves in the subtree that does not include leaves i and j as $Leaves_{ij}(e)$.





Edge Multiplicity Theorem: Given an additive matrix D , the multiplicity of an internal edge e is equal to -2 if e lies on $Path(i, j)$ in $Tree(D)$ and is equal to $-2 \cdot Leaves_{ij}(e)$ otherwise.

Proof: If an internal edge e lies on $Path(i, j)$ in $Tree(D)$, then e has a coefficient of $n - 2$ in $(n - 2) \cdot D_{ij}$ term in D^*_{ij} . To compute $Multiplicity_{ij}(e)$, consider the subtrees T_i and T_j formed by the removal of e . For each leaf k in T_i , $Path(j, k)$ passes through e , thus contributing 1 to the coefficient of e in $TotalDistance_D(j)$, but $Path(i, k)$ does not pass through e , thus contributing 0 to the coefficient of e in $TotalDistance_D(i)$.

Likewise, for each leaf k in T_j , $Path(i, k)$ passes through e , thus contributing 1 to the coefficient of e in $TotalDistance_D(i)$, but $Path(j, k)$ does not pass through e , thus contributing 0 to the coefficient of e in $TotalDistance_D(j)$. As a result, every leaf k will contribute 1 to the coefficient of e either in $TotalDistance_D(i)$ or in $TotalDistance_D(j)$. Thus, the coefficient of e in $TotalDistance_D(i) + TotalDistance_D(j)$ is n , which means that

$$Multiplicity_{ij}(e) = (n - 2) - n = -2.$$

On the other hand, if e does not lie on $Path(i, j)$, then e has a coefficient of 0 in $(n - 2) \cdot D_{ij}$. And if k is a leaf in the subtree not containing i and j , then to reach k , we must pass through e . Thus, the coefficient of e in each of $TotalDistance_D(i)$ and $TotalDistance_D(j)$ is equal to $Leaves_{ij}(e)$. It follows that

$$Multiplicity(e) = 0 - 2 \cdot Leaves_{ij}(e) = -2 \cdot Leaves_{ij}(e). \quad \square$$



We can interpret the Edge Multiplicity Theorem as stating that internal edges on the path $Path(i, j)$ have large multiplicities (-2) and other internal edges have small multiplicities (less than -2). Thus, if we are trying to minimize $D_{i,j}^*$ (among all possible choices of i and j), then we should look for a pair of leaves (i, j) having few internal edges on $Path(i, j)$. Neighbors have no internal edges connecting them, which makes them attractive candidates. The following exercise will get us part of the way toward proving that neighbors indeed minimize $D_{i,j}^*$.

Exercise Break: Show that if leaf i has a neighbor j in $Tree(D)$, and k is a leaf of $Tree(D)$ that is not a neighbor of i , then $D_{i,j}^* < D_{i,k}^*$.





Neighbor-Joining Theorem: Given an additive distance matrix D , a minimum element D^*_{ij} of the neighbor-joining matrix D^* must correspond to neighboring leaves i and j in $Tree(D)$.

Proof: Assume that D^*_{ij} is a minimum element of D^* but that i and j are not neighbors in $Tree(D)$. We aim to reach a contradiction by finding a pair of neighbors k and l such that $D^*_{kl} < D^*_{ij}$. By the preceding exercise, neither i nor j can have a neighbor if D^*_{ij} is a minimum element of D^* . Thus, i and j are the only leaves connected to $Parent(i)$ and $Parent(j)$, respectively. Because $Tree(D)$ is simple, $Parent(i)$ and $Parent(j)$ have degree at least equal to 3, meaning that each of these nodes is connected to at least two other nodes of $Tree(D)$, one of which lies on $Path(i, j)$. The other node is part of its own subtree; we call these subtrees T_1 and T_2 (see figure below).

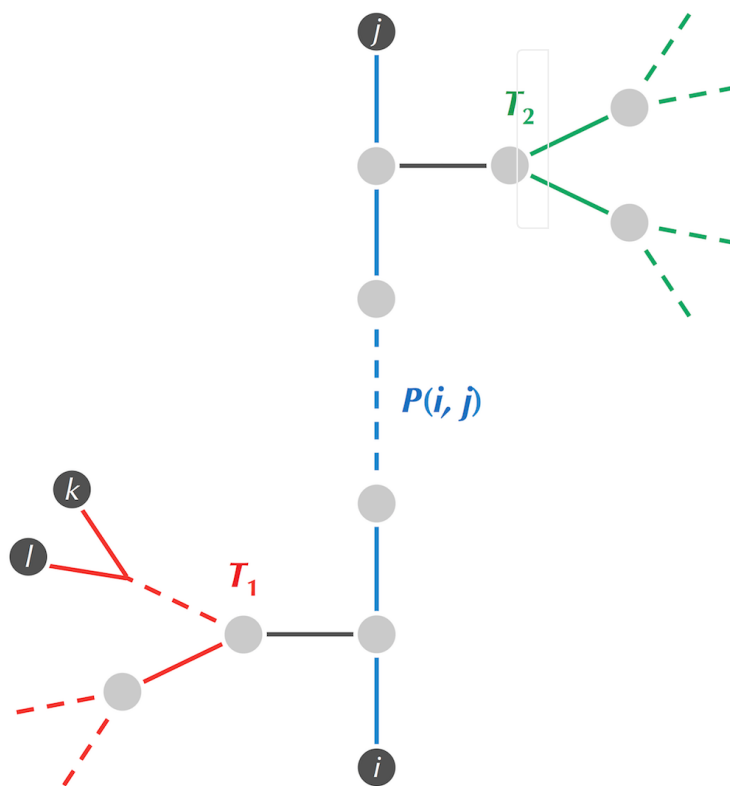
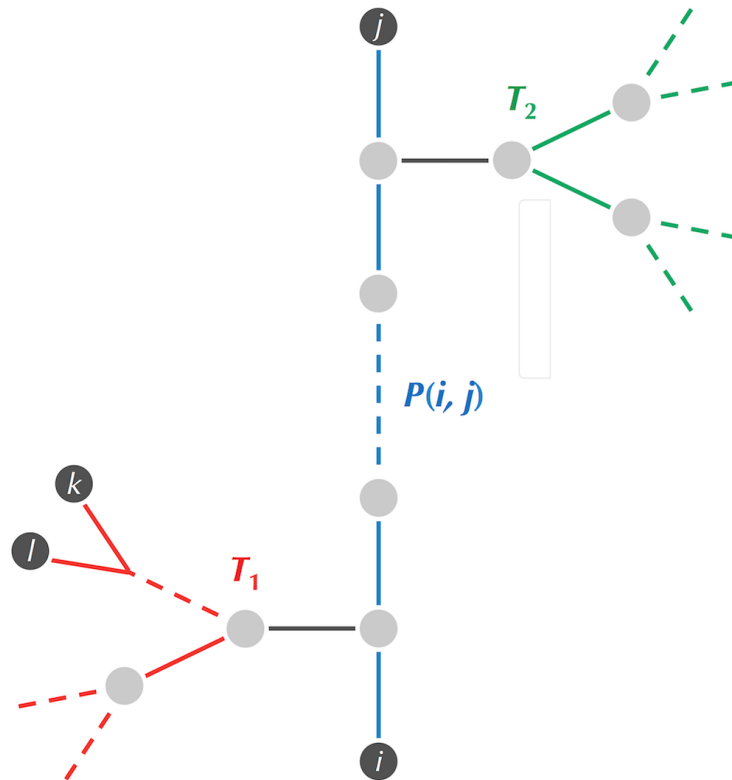


Figure: Leaves i and j are not neighbors. The unique path connecting them in $Tree(D)$, $Path(i, j)$, is shown in blue. Because $Tree(D)$ is simple, $Parent(i)$ and $Parent(j)$ must be connected to at least two other internal nodes, thus forming subtrees T_1 and T_2 (shown in red and green).

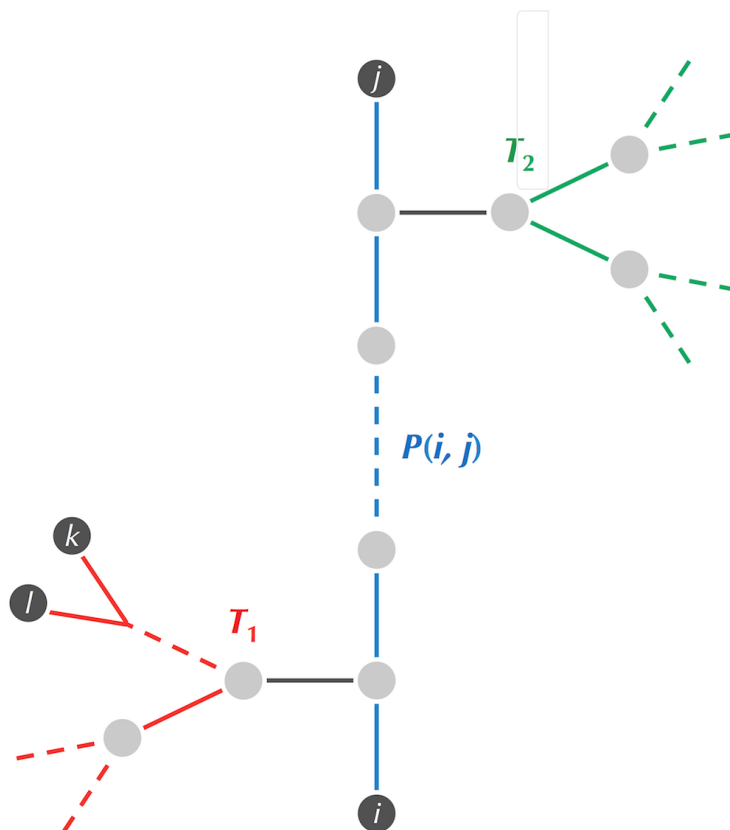


Without loss of generality, we will assume that the number of leaves in T_1 does not exceed the number of leaves in T_2 . Because i and j are not in T_1 or T_2 , T_1 must therefore contain fewer than $n/2$ leaves, and the rest of $Tree(D)$ must contain more than $n/2$ leaves (recall that n refers to the total number of leaves in $Tree(D)$). Because i has no neighbors, T_1 must have at least two leaves, which implies that T_1 has a pair of neighbors, which we denote as (k, l) . We will show that $D_{k,l}^* < D_{ij}^*$.



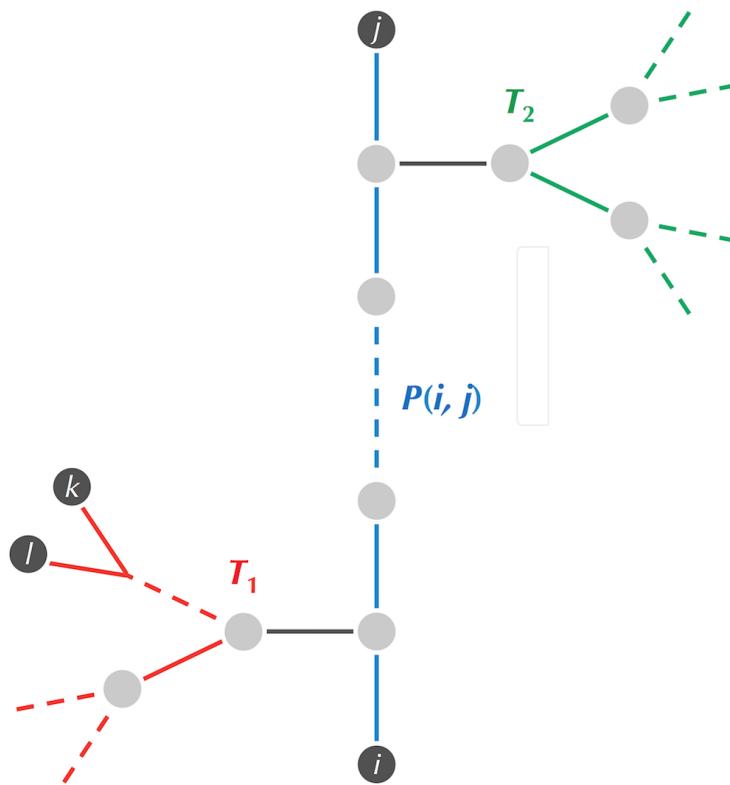
Consider an internal edge e of $Tree(D)$. We will first show that $D_{k,l}^* \leq D_{i,j}^*$ by showing that the multiplicity of e in $D_{k,l}^*$ does not exceed the multiplicity of e in $D_{i,j}^*$. There are three possibilities.

- If e lies on $Path(i, j)$, then by the Edge Multiplicity Theorem, $Multiplicity_{i,j}(e) = 2$, and the result follows.
- If e lies on $Path(i, k)$, then the removal of e breaks $Tree(D)$ into two subtrees, one containing k and l (with number of leaves equal to $Leaves_{i,j}(e) < n/2$), and the other containing i and j (with number of leaves equal to $Leaves_{k,l}(e) > n/2$). Thus, by the Edge Multiplicity Theorem, $Multiplicity_{k,l}(e) = 2 \cdot Leaves_{k,l}(e) < Multiplicity_{i,j}(e) = 2 \cdot Leaves_{i,j}(e)$.
- If e lies on neither $Path(i, j)$ nor $Path(i, k)$, then the subtree not containing i and j when e is removed is the same as the subtree not containing k and l . Thus, $Leaves_{i,j}(e) = Leaves_{k,l}(e)$, implying that $Multiplicity_{i,j}(e) = Multiplicity_{k,l}(e)$ (the Edge Multiplicity Theorem).





To prove that $D_{k,l}^*$ is in fact less than $D_{i,j}^*$, note that $Path(i, k)$ must contain an internal edge e because i and k are not neighbors. By the middle case above, we know that $Multiplicity_{k,l}(e) < Multiplicity_{i,j}(e)$. Our assumption that no internal edges of $Tree(D)$ have length zero means that d_e must be positive, which yields the result. $\square$





In the main text, we were attempting to assign limb lengths to leaves of a tree constructed from an arbitrary distance matrix. We set the limb length of i equal to $1/2(D_{ij} + \Delta_{ij})$ and the limb length of j equal to $1/2(D_{ij} - \Delta_{ij})$, where

$$\Delta = \frac{TotalDistance_D(i) - TotalDistance_D(j)}{n - 2}$$

Where do these formulas come from? Assume for a moment that D is an additive matrix, and select some leaf k that is not equal to i or j . If m is the parent of i and j , then we already have the limb length formula

$$LimbLength(i) = (1/2) (D_{ij} + D_{i,k} - D_{j,k})$$

As a result, it might seem like we should use this formula in the neighbor-joining algorithm for an arbitrary distance matrix D . However, if D is non-additive, then the expression $(D_{ij} + D_{i,k} - D_{j,k})/2$ will vary depending on how we select k . So we need a formula that still computes $LimbLength(i)$ when D is additive but that also gives us a single value when D is non-additive. To this end, we can compute the average of the above formula over all choices of $n - 2$ leaves k :

$$\frac{1}{n - 2} \sum_{\text{all leaves } k \text{ differing from } i \text{ and } j} \frac{D_{i,j} + D_{i,k} - D_{j,k}}{2}$$



If D is additive, then the sum above contains $n - 2$ terms that are all equal to $LimbLength(i)$. Furthermore, if D is non-additive, then this formula provides us with a unique value for estimating the limb length. Note also that the sum above has $n - 2$ occurrences of $D_{i,j}$. If we separate these out, then we obtain

$$\begin{aligned}
 LimbLength(i) &= \frac{D_{i,j}}{2} + \frac{1}{n-2} \cdot \sum_{\text{all leaves } k \text{ differing from } i \text{ and } j} \frac{D_{i,k} - D_{j,k}}{2} \\
 &= \frac{D_{i,j}}{2} + \frac{1}{n-2} \cdot \left(\sum_{\text{all leaves } k \text{ differing from } i \text{ and } j} \frac{D_{i,k}}{2} - \sum_{\text{all leaves } k \text{ differing from } i \text{ and } j} \frac{D_{j,k}}{2} \right) \\
 &= \frac{1}{2} \cdot \left(D_{i,j} + \frac{1}{n-2} \cdot \left(\sum_{\text{all leaves } k \text{ differing from } i} D_{i,k} - \sum_{\text{all leaves } k \text{ differing from } j} D_{j,k} \right) \right) \\
 &= \frac{1}{2} \cdot \left(D_{i,j} + \frac{(TotalDistance_D(i) - TotalDistance_D(j))}{n-2} \right) \\
 &= \frac{1}{2} \cdot (D_{i,j} + \Delta_{i,j})
 \end{aligned}$$

which is the formula in the main text that we used for computing $LimbLength(i)$.